Codako, A Video Game Construction Set

Constructivist learning with simulations

by
David Canfield Smith, Ben Gotow, Joyce Yuan
October 7, 2026

What is Codako?

Codako is a program that lets children ("of all ages" as Alan Kay used to say) create visual simulations. The goal of Codako is to improve the thinking skills of children. Since all video games are simulations, we present it to kids as a video game construction set. Yet the real value to kids lies in the nature of simulations, both in the simulations that they create and in the process of creating them.

Constructivism[1]

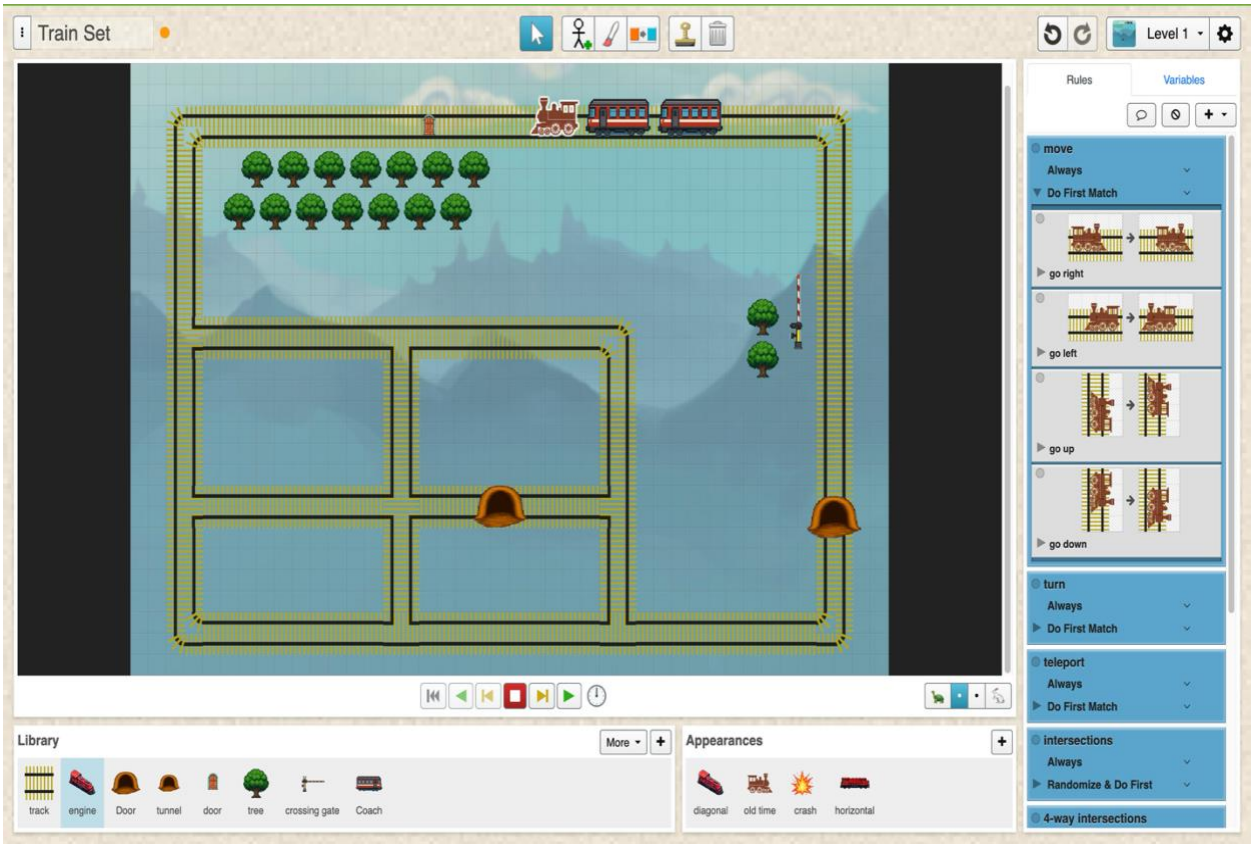
"Constructivism is a learning theory and educational philosophy asserting that learners actively construct knowledge rather than passively absorbing information. In contrast to traditional "instructivist" approaches—in which a teacher lectures and students act as empty vessels to be filled—constructivism views learning as an active, mental process of building new ideas on top of a foundation of prior experiences."[2] It works best when teachers provide kids with a solid foundation in a topic. But constructivism doesn't stop with the foundation; it *begins* there.

Codako supports constructivism. It encourages kids to learn by building games. Kids don't just play games built by others; they design and build the games themselves. This is intrinsically motivating for kids. By its nature it is a creative, active process. Children can't be passive when they use Codako.

Codako as a learning environment

Codako is optimized for building visual simulations. Unlike animations, simulations don't run the same way every time. They depend on inputs, usually provided by users. In a game context, the ability to make inputs is essential to playing the game. More broadly though, simulations let learners conduct "what if?" experiments, test ideas, hone critical thinking skills, and achieve a deeper understanding of complex topics. They provide a rich learning environment.

Codako uses a theatrical metaphor. The display screen is called the “stage.” Objects on the stage are called “characters.” The simulation as a whole is called the “game” or the “world.”



The Codako authoring window, showing (counterclockwise) the stage, the characters in the game (the Library), the available appearances for the selected character (in this case the engine), and the rules for that character.

In a new world, the stage is empty. Kids populate it with the Create a Character tool. They specify how they want characters to look with the Draw an Appearance tool. They give them behavior with the Make a Rule tool. These three tools form a “mantra”:

- Create a Character
- Draw an Appearance
- Make a Rule

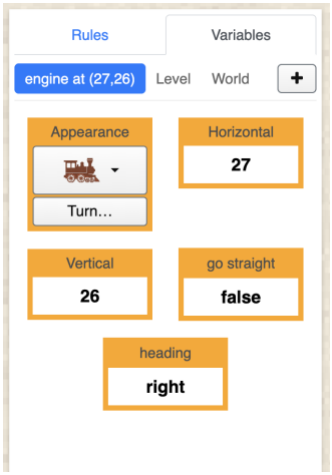
This is how kids solve problems in Codako—repeat the mantra. Codako has a simple clean mental model. It applies a small number of concepts consistently.

Codako as a programming language

Codako is based on object-oriented programming (OOP). It makes innovative use of the theatrical metaphor to present OOP principles[3]. In the theater, the script defines characters (classes), e.g. soldiers, and the characters are played by one or more actors (instances of

the class). The play takes place on a stage (2D grid), and each actor has its own appearance and state during the play.

Characters in Codako are full objects in the OOP sense. They are not dummied down. They have both state (variables) and behavior (rules).



State variables for a train engine

Rules exist at the character (class) level, and state exists at the actor (instance) level. The theater metaphor enables kids to get started quickly ("low floor") and grows with them as they master concepts like variables, appearances, and multiple stages ("high ceiling"). When it introduces the idea that variables and rules *belong to an object*, the basis of OOP, the analogy between theater and OOP makes it seem natural.

Codako teaches all the important programming concepts. Kids have reported that when they learned other programming languages like Basic or Java, they already understood the principles.

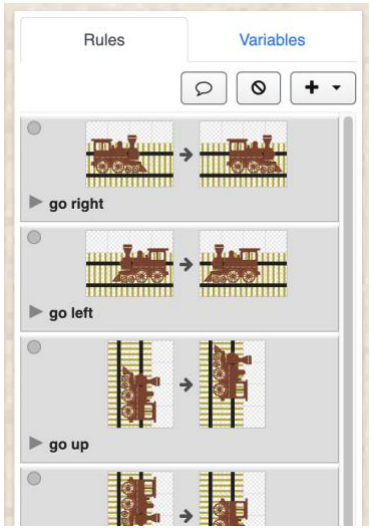
The user interface to programming

But Codako goes further than just providing the capabilities of OOP. *It improves the user interface to programming*. It employs three powerful innovations that separate it from other programming languages:

1. Programming without a programming language
2. Programming by demonstration
3. Detail suppression

☆ 1. Programming without a programming language[4]

By this we mean a syntactic language that uses computer terminology. This is the biggest difference. A Codako program consists of a set of visual before-after rules:



A few rules for a train engine

Each rule has two pictures connected by an arrow. Each picture portrays a small portion of the simulation—the area around the character having the rule. The left side shows what that portion must look like in order for the rule to execute. The right side shows what that portion will look after the rule executes. You may read it as "IF the left side matches the simulation environment, THEN modify the simulation to look like the right side.

Rules in Codako are equivalent to procedures in a conventional programming language. But there is no programming language syntax: no begin-end, if-then-else, x := f(y), etc. Instead rules involve only images in the game the kids are making. Thus rules operate in the kids' conceptual space: the "cast of characters on the stage." At no time do kids have to type a single line of computer code or learn computer terminology.

☆ 2. Programming by demonstration[5,6,7]

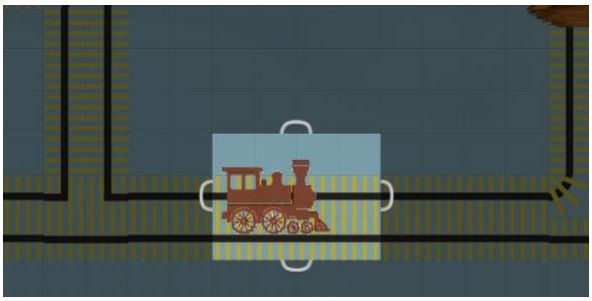
So how do kids specify what a rule should do? Instead of typing a description of the desired actions, they simply perform the actions themselves. The system remembers what they did and reproduces it on demand. For example, if kids want a train engine to move along a track, they simply grab the engine and drag it to where they want it to go:



Drag and drop, the technique used in every personal computer on Earth

Codako allows additional conditions to be specified in the left side of a rule. These conditions typically test the state (variables) of the characters in the rule. Like the picture, the conditions must be satisfied in order for the rule to match.

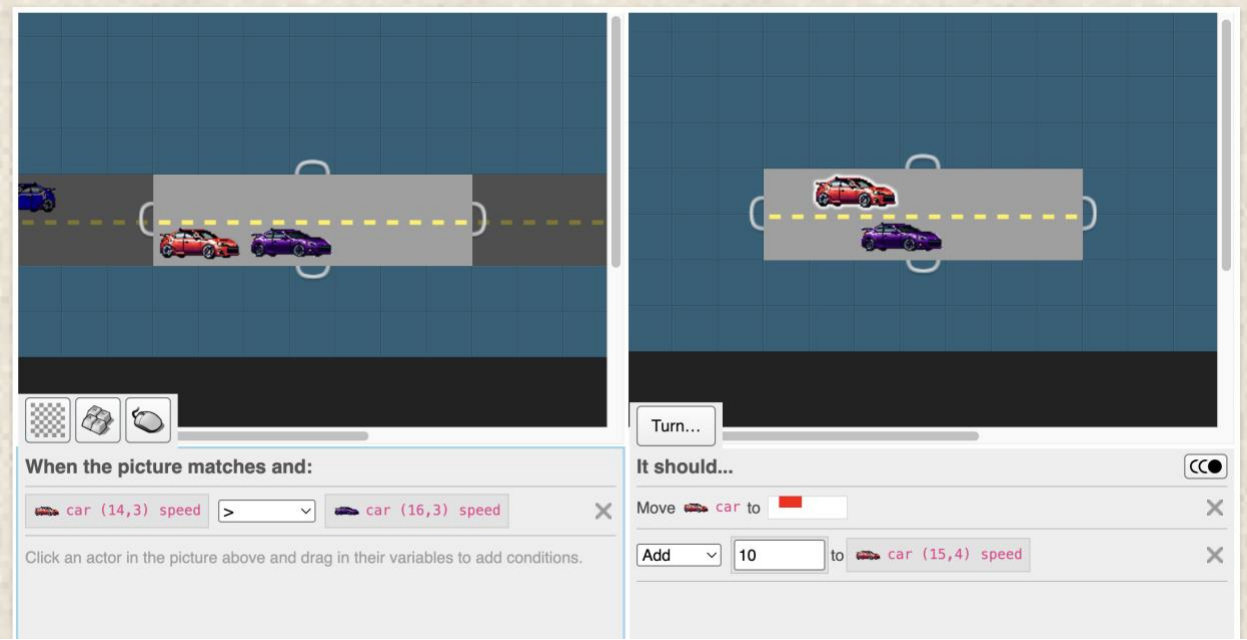
Kids specify the region of a game that the rule can deal with using what is called the "spotlight" (following the theatrical metaphor). It is like the light that shines on an actor on the stage to attract the audience's attention. When a rule is being defined, the stage is darkened except for a bright spot around the character getting the rule:



Spotlight showing a rule being defined for the train engine

Kids can adjust the size of the Spotlight with the handles on the sides. This bounds the domain of the rule; it can only modify things inside the Spotlight.

Kids can create, delete, and move the characters in the Spotlight. This is all done by direct manipulation—drag and drop; there is no intermediate language. For example, in defining the rule below a kid has dragged the red car into the passing lane to initiate passing another car. The action is duly recorded in the right side of the rule.



A passing rule for cars

In addition, kids can test and change the variables of characters in the Spotlight. Tests appear in the left side of the rule, changes in the right side. So the elements of this rule are:

- The picture says: "if there is a car directly in front of me (the red car), and there are no cars coming the other way."
- The test under the picture says: "if I am going faster than the other car." (There can be any number of such tests.)
- The actions say: "move me into the passing lane and speed me up." (Again, there can be any number of such actions.)

Since the appearance of the cars is not tested, this rule will apply to all colors of cars, i.e. all instances of class "car." Thus all cars can pass all other cars with this one rule. However, by introducing additional conditions such as checking for specific variable values, the applicability of the rule can be reduced if desired, all the way down to a single character instance.

Of course this rule doesn't work so well for 2-lane roads. It doesn't account for cars coming the other way that may be just outside the Spotlight. There may not be time to pass. It will need refining. But that is the *very point* of simulations. Kids can experiment, try things out, make mistakes, learn by doing. Codako supports Elon Musk's engineering philosophy:

build fast, test fast, fail fast, learn fast, repeat.

☆ 3. Detail suppression

A rule may have dozens of conditions and actions in it, making it a fairly complex algorithm. But Codako has an innovative way of dealing with this. Regardless of its complexity, a rule is always portrayed as a single before and after picture, showing the starting and ending states. This is the rule's meaning at a glance. The details are hidden until kids ask to see them. Kids have little trouble understanding what a rule does by identifying the differences between the two pictures. We've seen kids go down long list of rules explaining what each one does, even rules written by other kids in other schools for other games.

People are good at comparing two images side by side and saying how they differ. And they enjoy doing it; it is a popular item in comic strips:



What they are not good at is looking at long sequences of computer instructions and building a mental model of the effect of those instructions. George Miller in his famous paper *The Magical Number 7±2* showed that most people can only retain 5-9 chunks of information in their short term memory at a time.[8] More than that and their memory "overflows" and starts discarding chunks to make room. This is the single most important reason why kids can't write large programs in conventional programming languages: those languages display *all the steps, all the detail, all the time*. Kids (and ordinary people) drown in the complexity. Codako's detail suppression allows kids to build complex programs, not just simple ones.

How Codako executes simulations

Codako uses a simulation execution engine consisting of:

- **a clock** – A simulation clock advances in discrete steps. The clock may be sped up, slowed down, stopped, or even run backward (useful for debugging).
- **a pattern matcher** – On each clock tick, Codako goes through all the characters on the stage, giving each one a chance to run. For each character, Codako runs down its rules testing each one. If a rule matches the character's neighborhood on the stage, Codako executes it. Otherwise it skips the rule and tries the next one. Every character gets to execute only one rule in each clock cycle (although ways exist to allow it to execute more than one). If none of a character's rules execute, then the character does nothing in this

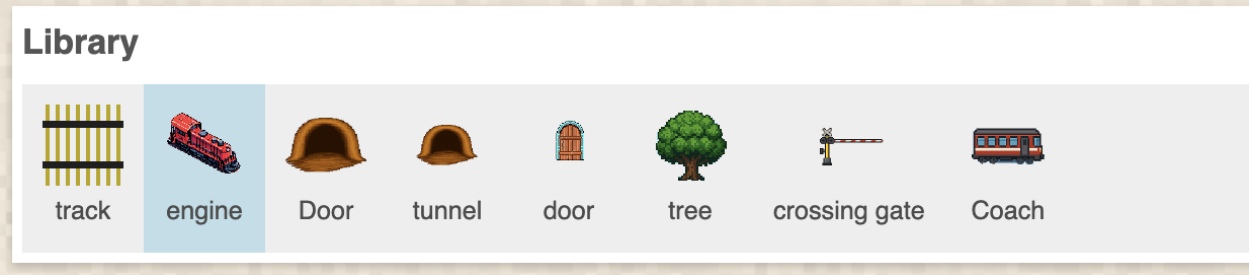
clock cycle. Codako goes on to the next character. When all the characters on the stage have had a chance to run, the clock advances one tick, and the process begins again.

- **loops** – Note that looping is an inherent part of the execution engine. The rules are tested in every clock cycle. Thus if a rule does nothing more than move a character one space, it will enable the character to move across the entire screen, one clock cycle at a time.
- **conditionals** – Likewise conditionals are an inherent part of the execution engine. Every rule is a conditional: IF [the left side of a rule matches] THEN [execute the right side].

Conditionals and loops are difficult concepts for kids. By making them part of the execution engine, Codako relieves kids from having to deal with them explicitly.

☆ Remaining in the user's conceptual space

Codako's rules deal with objects and concepts that kids make: trains and track and intersections and crossing gates:



Not machine concepts: bits and bytes and arrays and, most of all, syntax.

A bit of history: When Xerox scientists including one of us (Smith) were designing the Xerox Star computer (the ancestor of the Macintosh) in the 1970s, we started with two strikes against us. Our target users — "knowledge workers" in an office — (a) didn't understand computers and (b) were fearful of them. They were resistant to having to use them. That's when we came up with the key idea: instead of trying to take office workers into the computer's world, we would bring the computer into the office world. Smith's invention of icons was the perfect vehicle for modeling objects and actions in the office, such as documents, folders, printers, wastebaskets. This came to be known as the "desktop metaphor." It was a smashing success, was easy to use, was readily accepted by office workers, and has been the basis for all subsequent personal computer user interfaces.

Codako adopted this approach for programming. Instead of trying to take kids into the computer's world, it brings programming into the kids' world. To program in Codako, kids don't learn an alien language of computer terminology and concepts; they simply manipulate the objects they create.

The history of Codako

Cocoa – Codako began as a research project in the Advanced Technilogy Group (ATG) at Apple Computer in the 1990s. At the time Apple was actively engaged in assisting education. One of us (Smith) along with Allen Cypher and Alan Kay (the father of personal computers) took on the challenge of enabling kids to program computers. The first prototype was called Cocoa. Here is 11 year old Gregory Miller (the son of one of the editors of this book) demoing Cocoa to the Apple community in 1996.[10]

KidSim – Later the three of us with help from ATG researchers produced a more refined and capable version called KidSim (for Kids' Simulations). We did extensive testing of it with kids in schools, in collaboration with several progressive classroom teachers. Kids built many games in it. Even teachers built some simulations to illustrate lessons. We verified to our satisfaction that KidSim worked for "children of all ages."

Creator – A group of us working on Kidsim spun off from Apple in the late 90s and founded a startup called Stagecast Software. Our goal was to turn KidSim into a product which could be used by anyone. Which we did. But the market in educational software back then was difficult. There were no tablet computers or smartphones or even AppStores. It was a competition for shelf space in electronics stores. We ran out of money and had to close up shop.

Codako – While it was still available, one of us (Gotow) used it as a child. He fell in love with it, which contributed to his desire to make programming his profession. He did, and today he runs his own software startup. But he never forgot Creator. A couple of years ago he decided to resurrect it. Working on his own in his free time, nights and weekends, without pay, he rebuilt the core of it strictly from memory. That's when the rest of us discovered his project and asked to join it. Together we have not only reproduced everything that was important in Creator but drastically improved its user interface as well. Codako is written in Javascript using modern (20 years later) tools and API's. It runs in any web browser. We will release it to the world soon.

Codako versus AI

Traditionally, kids would write a game by learning a machine's language and writing it in that language (code). With the advent of LLMs and agentic AI, kids can now write games just by expressing what they want in plain English! So why do we even need Codako? Both of these extremes are poor learning environments. Learning to code has a high initial cost and a minimum age requirement. Using AI has nearly zero cost but completely bypasses the machine's concepts, removing opportunities to build a mental model of the system. Problem solving fundamentally requires two things: a goal and an understanding of the tools at your disposal. Codako's 50-year-old approach—programming by demonstration—is an ideal compromise between these two extremes. Kids demonstrate what to do in their language, and they see the machine's interpretation in its language. "I moved this character to the right, and you recorded X=X+1". As

kids interact with the simulation builder, they gain an understanding of the system, the various tools and objects at their disposal, and creativity takes hold!

The true test for education is "what kinds of ways should the gamut of children undergoing this process be able to think-about/do afterwards?" —Alan Kay

What have they learned? What skills did they develop? And just as important, did they enjoy the experience? Codako tries to make learning really fun and focused.

"Younger audiences, creators, and lifestyle media experience technology differently. They do not want only a specification sheet. They want context. They want emotion. They want something they can share, discuss and remember." —Leif Lindner, CEO of IFA Management

It is revealing that after Greg Miller's team built a game in Cocoa illustrating the life cycle of water[10], they knew not only how the simulation worked but that they could modify it to introduce acid rain. This demonstrates that they understood the *meaning* of what the simulation was modeling, not just its *behavior*.

In Alan Kay's terms, "In other words, making an animation of a bouncing ball doesn't count for much in this context, but just two lines of code can easily portray gravity as a 'velocity eater,' and thus a 10 year old can construct a decent/real Newtonian model (and furthermore can get the actual relations directly from nature). With regard to 'AI', having a musical instrument that will play the notes for the player doesn't help the player get more musical. Having a bat that hits the ball automatically doesn't help a kid learn baseball. If you get the 'AI' out of the centre of things, then there are many opportunities to support internal learning by the child for many subjects. To do this our culture has to quit overvaluing results and start valuing *processes and human growth* (this is a tough sell these days)."

"If the goal is simply to have a finished game to play, an LLM makes Codako obsolete. But if the goal is *human learning, empowerment, and cognitive growth,* a tool like Codako offers things an LLM inherently cannot." —Gemini

Thinking skills taught by Codako

☆ **Visual thinking –** The Sapir-Whorf Hypothesis holds that the language we speak determines how we think. Wittgenstein once famously said, "The limits of my language mean the limits of my world;" i.e. our conceptual universe is bounded by our linguistic system. Learning another language can rewrire our brains, expanding our minds by focusing our attention on different aspects of our environment. Codako gives kids a new language in which to think, a visual one of before-after images. It stimulates the right (image processing) side of the brain, supplementing the left (symbolic) side that schools usually focus on. It engages kids' entire brains in the problem solving process.

☆ **Sequential thinking** – Designing a game inherently involves planning sequences of steps for the players. In order to achieve W, they must first achieve X and Y and Z. This is especially true for a game that has multiple levels. Thinking sequentially is inherent in game design.

☆ **Critical Thinking** – You can't debug an algorithm without thinking deeply and objectively about what's going on and why. There's no room for biases or personal beliefs, only the truth.

☆ **The scientific method** – When scientists encounter something they don't understand, they form a hypothesis about what is happening. This is the creative part of science. Then they formulate an experiment to test that hypothesis. This is the artistic side of science; some of the experiments that scientists have designed are truly beautiful. Then they execute the experiment. Either their hypothesis is confirmed, or it is refuted. If it is refuted, it's back to square one. Even if it's confirmed, it doesn't mean that the hypothesis is true in general. It's just true in this case. Scientific theories are always just one experiment away from becoming invalid. Yet nothing in human history has led to more progress than the scientific method.

☆ **Constructivism** – Codako emphasizes learning by building things, as we discussed earlier. It is interactive learning.

☆ **The act of creation** – The activity of programming computers encompasses all of the above thinking skills, plus one additional one: the reward of *seeing your ideas come to life*. This is one of humanity's deepest desires and has motivated artists throughout history. It is the lure that has led many people—including the present authors—to become programmers.

Conclusion

Codako is a powerful learning tool disguised as a video game construction set. It incorporates innovative techniques developed through many years of trial and error. *Programming by demonstration* lets kids get started building games without the overhead of learning a technical language. *Visual before-after rules* let them understand how the games work. *Detail suppression* lets them build larger games.

But video games are simulations. In reality kids are becoming proficient in understanding and using simulations. Along the way they are learning thinking skills that include *sequential thinking, visual thinking, critical thinking,* and the *scientific method*. They are learning to *program computers*, using powerful methods like object-oriented programming. Most of all they are engaging in constructivist learning, *learning by building things*, actively participating in the learning process. This nurtures a fundamental element of human nature: *the joy of creation.*

References

1. McLeod, S. *Constructivism Learning Theory & Philosophy of Education*, simplypsychology.org/constructivism.html, (March 2025).

2. Nice summary by Gemini, Google's LLM.

3. Booch, G., Maksimchuk, R., Engle, M., Young, B., Conallen, J., and Houston, K. *Object-Oriented Analysis and Design with Applications*, 3rd edition, Addison-Wesley Professional, 2007.

4. Smith, D.C., Cypher, A., and Spohrer, J. *KidSim: Programming Agents without a Programming Language*, CACM 37,7 (July 1994), 54-67.

5. Smith, D. C. *Pygmalion: A Computer Program to Model and Stimulate Creative Thought*, Birkhäuser Verlag, 1977. Also *Pygmalion: A Creative Programming Environment*, PhD. Dissertation, Stanford University, 1975.

6. Cypher, A. (Ed.) *Watch what I do: Programming by demonstration*, MIT Press, 1993.

7. Lieberman, H. (Ed.) *Your wish is my command: Programming by example,* Morgan Kaufmann Publishers, 2001.

8. Miller, George, *The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information*, Psychological Review 63,2 (March 1956), 81–97. https://doi.org/10.1037/h0043158

9. Smith, D.C., Irby, C., Kimball, R., Verplank, W., and Harslem, E. *Designing the StarUser Interface*, Byte 7,4 (April 1982), 242-282.

10. Miller, Gregory, Demo of Cocoa at Apple's Worldwide Developers' Conference, 1996. https://www.youtube.com/watch?v=NtmGTrr3R-E